Actividad 5 - Introducción a las Funciones con Meta AI

Sergio Sánchez Reyes

Tipología de funciones en Python

Actúa como tutor de Python para Ciencia de Datos. Explícame detalladamente en qué se diferencian los distintos tipos de funciones en Python (por ejemplo: built-in, definidas por el usuario con def, lambda/anónimas, métodos (instancia/clase/estático), generadoras con yield y funciones async). Para cada tipo: (1) definición breve, (2) un ejemplo corto en Python, y (3) un caso real de cuándo usarla.

¡Claro que sí! Con gusto te explico los distintos tipos de funciones en Python, enfocándome en su utilidad para la Ciencia de Datos. ¡Vamos a desglosarlos!

1. Funciones Built-in (Integradas)

Son funciones que ya vienen predefinidas en Python y están disponibles para usar de inmediato sin necesidad de importarlas. Son la base de muchas operaciones.

- **Definición breve:** Funciones que Python provee de fábrica para tareas comunes.
- **Ejemplo:**

```python
lista_numeros = [10, 20, 30, 40, 50]
longitud = len(lista_numeros)
print(longitud) # Output: 5
```

3 lines (3 loc) · 106 B

- **Caso de uso en Ciencia de Datos:** Son omnipresentes. `len()` para saber el tamaño de un dataset o una columna, `sum()` para calcular la suma de valores, `min()`/`max()` para explorar rangos de datos, `print()` para depurar, `type()` para verificar el tipo de una variable, etc.

2. Funciones Definidas por el Usuario (`def`)

Estas son las funciones más comunes. Las creamos nosotros para encapsular lógica específica y hacer

Arquitectura y diseño de funciones

Actúa como ingeniero/a de software. ¿Bajo qué criterios técnicos es preferible que una función definida por el usuario reciba parámetros o no? Incluye aspectos como acoplamiento, pureza, dependencias, configuración, testabilidad y reutilización. Además, ¿en qué escenarios es mejor retornar un valor frente a solo ejecutar una acción con efectos secundarios? Da ejemplos concretos en Python.

¡Claro! Como ingeniero de software, te diré que la decisión de si una función definida por el usuario debe recibir parámetros o no es fundamental y afecta directamente la calidad, la mantenibilidad y la escalabilidad de tu código. Vamos a desglosar los criterios técnicos clave:

Criterios Técnicos para Recibir o No Parámetros

La regla general es simple: **una función debe recibir parámetros para cualquier dato que necesite y que no sea una constante universal o una dependencia gestionada a través de inyección.**

Aquí te explico los criterios en detalle:

1. **Acoplamiento (Coupling):**
   - **Con parámetros:** Reduce el acoplamiento. La función es "autónoma" respecto a cómo obtiene sus datos de entrada. No necesita saber cómo se generan esos datos, solo que se le proporcionan. Esto permite cambiar el origen de los datos sin modificar la función.
   - **Sin parámetros (dependencia interna):** Aumenta el acoplamiento. Si una función accede directamente a variables globales, configuraciones cargadas de forma implícita o a recursos específicos (como una base de datos o un servicio externo) sin que estos se pasen como parámetros, se acopla fuertemente a su entorno. Cambiar la forma en que se obtienen esos datos requerirá modificar la función misma.

   Ejemplo Python:

```python
# Acoplamiento alto (mal):
# 'data_global' es una dependencia implícita
data_global = [1, 2, 3]
```

UDF vs. funciones lambda

Compara funciones definidas por el usuario (UDF con def) vs funciones lambda en Python. ¿En qué situaciones específicas es preferible usar def en lugar de lambda? Explica el impacto en legibilidad, depuración, documentación, pruebas y reutilización. Incluye un ejemplo con lambda y su refactor a def (y explica por qué mejora)

¡Absolutamente! Como ingeniero de software, te diré que la elección entre una función `def` y una `lambda` no es trivial y tiene implicaciones significativas en la calidad de tu código. Vamos a desglosarlo.

Comparación General: `def` vs. `lambda`

Característica	`def` (Función Definida por el Usuario)	`lambda` (Función Anónima)
Sintaxis	`def nombre_funcion(parametros): <bloque de código>`	`lambda parametros: expresion`
Nombre	Sí, tienen un nombre (`nombre_funcion`).	No, son anónimas (aunque se pueden asignar a una variable).
Cuerpo	Pueden contener múltiples expresiones y sentencias (bucles, condicionales, etc.).	Solo una única expresión. El resultado de la expresión es el valor de retorno.
Docstrings/Comentarios	Permiten docstrings y comentarios internos para explicar su funcionalidad.	No soportan docstrings ni comentarios de forma estándar.
Versatilidad	Muy versátiles, para cualquier lógica compleja.	Limitadas a lógica muy simple y concisa.
Recursión	Sí.	Posible, pero no idiomático ni recomendable.

¿En qué situaciones específicas es preferible usar `def` en lugar de `lambda`?

La regla general es: **Usa `def` casi siempre.** Las lambda tienen un nicho muy específico.

Conclusión:

La conclusión a la que llegué es que, las funciones son la base para construir código reproducible y listo para producción en cualquier proyecto. Por eso, es fundamental entender cómo funcionan y esforzarnos por escribir código fácil de mantener. Así, en el peor de los casos, si surge un problema, podremos depurarlo sin mayores complicaciones.